



MONAR

Mapa de Viajes Social con Globo 3D Interactivo

PHP 8.4

MySQL 8.0

Globe.gl

Docker

Symfony 8

TopoJSON

Vanilla JS

World-Atlas

Guía completa de tecnologías, arquitectura y preguntas de defensa

Trabajo de Fin de Grado · 2026

Índice

- 1 **Stack tecnológico completo** – todas las librerías, frameworks y APIs
- 2 **Esquema de base de datos** – tablas, relaciones y justificación
- 3 **Funcionamiento de la web** – de principio a fin, flujo completo
- 4 **Arquitectura del proyecto** – estructura de ficheros y decisiones de diseño
- 5 **Seguridad implementada** – medidas contra las OWASP Top 10
- 6 **Preguntas de defensa del TFG** con respuestas argumentadas

1. Stack Tecnológico Completo

Backend – Servidor y Lenguaje

**PHP 8.4**

v8.4

Backend

Lenguaje principal del servidor. Se utiliza PHP nativo (sin framework MVC para el routing) para todos los endpoints de la API REST: autenticación, gestión de países visitados, fotos, reseñas, likes, seguidores y panel de administración.

¿Por qué PHP y no Node.js/Python? PHP está diseñado específicamente para web, tiene soporte nativo en servidores Apache, su rendimiento es más que suficiente para el volumen de datos del proyecto y la integración con MySQL mediante PDO es muy directa. Además, la imagen Docker `php:8.4-apache` incluye todo lo necesario. PHP 8.4 trae mejoras de rendimiento (JIT compiler) y tipado estricto moderno.

**PDO (PHP Data Objects)**

nativo PHP

Backend Seguridad

Capa de abstracción para acceder a la base de datos MySQL. Toda la lógica de negocio usa PDO con **prepared statements** (sentencias preparadas) para comunicarse con MySQL.

¿Por qué PDO y no mysqli? PDO es agnóstico del motor de base de datos (si en el futuro se cambia a PostgreSQL, solo cambia el DSN). Sus prepared statements previenen de forma automática inyección SQL — el parámetro nunca se concatena directamente en la query. Se configura con `PDO::ATTR_EMULATE_PREPARES = false` para que las consultas sean realmente preparadas en el servidor MySQL.

**Sesiones PHP nativas**

Backend Seguridad

Sistema de autenticación basado en `session_start()` + `$_SESSION`. Cada página PHP y cada endpoint de la API comprueba `$_SESSION['user']` antes de procesar nada.

¿Por qué sesiones y no JWT? JWT requeriría un sistema de refresh tokens, almacenamiento seguro en cliente y gestión de revocación. Para una aplicación web PHP tradicional donde todas las páginas

se sirven desde el mismo servidor, las sesiones son la solución estándar, más simple y con menor superficie de ataque.



password_hash() / password_verify()

nativo PHP

Seguridad

Todas las contraseñas se almacenan como hash con `password_hash($password, PASSWORD_DEFAULT)` que usa **bcrypt** con salt aleatorio automático. La verificación se hace con `password_verify()`, nunca comparando el hash directamente.

¿Por qué bcrypt y no MD5/SHA1? MD5 y SHA1 son funciones de hash de propósito general, rápidas y por tanto vulnerables a ataques de fuerza bruta. Bcrypt es un algoritmo de hash lento por diseño (work factor ajustable) y resistente a GPU cracking. `PASSWORD_DEFAULT` garantiza que si PHP actualiza el algoritmo recomendado, las contraseñas nuevas usarán el mejor disponible.



Symfony 8.0

v8.0.*

Backend

Symfony **no** gestiona el routing ni las peticiones HTTP del día a día (eso lo hace PHP nativo). Se usa específicamente para:

- **Doctrine Migrations:** sistema de control de versiones del esquema de la base de datos
- **Doctrine ORM:** define las entidades y sus relaciones para generar las migraciones automáticamente
- **Symfony Console:** el comando `php bin/console doctrine:migrations:migrate` se ejecuta al arrancar el contenedor Docker

¿Por qué Symfony si no se usa como framework MVC? Se eligió su sistema de migraciones porque es el más robusto del ecosistema PHP. Permite aplicar cambios de esquema de forma reproducible y sin perder datos, lo que es esencial en un entorno Docker donde la base de datos se recrea. El resto del proyecto usa PHP nativo para mayor control y simplicidad.



Doctrine ORM + Doctrine Migrations Bundle

^3.6 / ^4.0

Backend

- **Doctrine ORM:** mapeo objeto-relacional. Define las entidades PHP que representan las tablas (Usuario, Pais, VisitaPais, Publicacion, Resena, Likes, etc.)
- **Doctrine Migrations:** genera y ejecuta ficheros de migración SQL (`Version20260413122830.php`) que crean/modifican las tablas en MySQL de forma versionada y reversible

¿Por qué migraciones en lugar de un script SQL manual? Un script SQL hay que ejecutarlo manualmente y puede fallar si ya existe parte del esquema. Las migraciones trackean qué se ha aplicado, son idempotentes, y se ejecutan automáticamente en el arranque del contenedor Docker.

Base de Datos



MySQL 8.0

v8.0Base de Datos

Motor de base de datos relacional. Almacena usuarios, países, visitas, fotos, reseñas, likes y relaciones de seguimiento. Todas las tablas usan motor **InnoDB** con soporte de transacciones, claves foráneas y charset utf8mb4 (soporte completo de emojis y caracteres internacionales).

¿Por qué MySQL y no PostgreSQL o MongoDB? Los datos del proyecto son relacionales (usuarios tienen visitas, visitas tienen fotos, fotos tienen likes). MySQL 8.0 tiene excelente soporte en el ecosistema PHP, la imagen Docker oficial es estable, y para este volumen de datos su rendimiento es idóneo. PostgreSQL sería equivalente; MongoDB no encajaría bien con las relaciones entre entidades.



Servidor Web



Apache 2.4 con mod_rewrite

php:8.4-apacheInfraestructura

Servidor HTTP que sirve los archivos PHP y los assets estáticos. mod_rewrite está habilitado para permitir URLs limpias. El DocumentRoot apunta a /var/www/html/public, de modo que solo se expone la carpeta pública.

¿Por qué Apache y no Nginx? La imagen Docker oficial php:8.4-apache incluye PHP y Apache preconfigurados, lo que simplifica enormemente el Dockerfile. Nginx requeriría php-fpm como proceso separado y configuración adicional.



Infraestructura y Despliegue



Docker + Docker Compose

v3Infraestructura

El proyecto se ejecuta con dos contenedores orquestados por docker-compose.yml:

- **monar_app**: imagen `php:8.4-apache` + Composer + código PHP → puerto 8080
- **monar_db**: imagen `mysql:8.0` → puerto 3306 con volumen persistente

El contenedor de PHP espera a que MySQL esté listo (healthcheck) antes de ejecutar las migraciones y levantar Apache.

¿Por qué Docker? Garantiza que cualquier persona pueda ejecutar el proyecto con `docker compose up` sin instalar PHP, MySQL, ni Composer localmente. Elimina el problema de "en mi máquina funciona". El volumen Docker persiste los datos de la base de datos entre reinicios.



Composer 2 v2

Herramienta

Gestor de dependencias PHP. Descarga e instala automáticamente Symfony, Doctrine y todos los paquetes de `composer.json`. Se ejecuta dentro del contenedor Docker durante el arranque (`docker-entrypoint.sh`).

Estándar de facto en el ecosistema PHP. Sin Composer, habría que gestionar manualmente el autoloading y las dependencias de cada paquete.



Frontend – Librerías JavaScript

**Globe.gl**

última vía unpkg CDN

Frontend CDN

Librería JavaScript para renderizar un **globo terráqueo 3D interactivo** en el navegador. Usa internamente Three.js (WebGL) para el renderizado. Se usa para:

- Mostrar el planeta Tierra con textura Blue Marble
- Pintar polígonos de países sobre el globo (visitados en morado, no visitados en gris oscuro)
- Detectar clicks sobre países
- Rotación automática, zoom y arrastre
- Efectos de atmósfera

```
// Cargado vía CDN <script src="https://unpkg.com/globe.gl"></script> // Uso
en map.js const globe = Globe() (el) .globeImageUrl('/unpkg.com/three-
globe/example/img/earth-blue-marble.jpg') .atmosphereColor('#7f5cff')
.polygonsData(countries) .polygonCapColor(feature => isVisited ? '#4c4a7a' :
'#2a2f3f')
```

¿Por qué Globe.gl y no Leaflet o Google Maps? Leaflet y Google Maps son mapas 2D planos. Globe.gl ofrece un globo 3D que es el elemento visual distintivo y más atractivo de MONAR. La diferenciación visual respecto a otras apps de tracking de viajes es precisamente el globo interactivo. Además es de código abierto y gratuito.

**TopoJSON (Client)**

vía Globe.gl

Frontend

Librería para parsear archivos TopoJSON y convertirlos en GeoJSON. Globe.gl lo incluye internamente. Se usa con `topojson.feature(worldData, worldData.objects.countries)` para extraer los polígonos de países del dataset World-Atlas.

TopoJSON es un formato más compacto que GeoJSON (comparte aristas entre polígonos), lo que reduce el tamaño del archivo de datos geográficos del mapa mundial.



APIs y Recursos Externos

**World-Atlas @2 (unpkg CDN)**

v2

API / Dataset

Dataset TopoJSON con los polígonos geográficos de todos los países del mundo a escala 1:110 millones. Se descarga en tiempo real al cargar el mapa:

```
fetch('https://unpkg.com/world-atlas@2/countries-110m.json')
```

Cada país tiene un ID numérico ISO 3166-1 que se cruza con los códigos ISO almacenados en la base de datos para saber qué países pintar como "visitados".

¿Por qué este dataset? Es el estándar de facto para visualizaciones geográficas en web (es el mismo que usa D3.js). Es de dominio público, gratuito, y el formato `countries-110m.json` (resolución 1:110M) es perfecto para un globo 3D donde no se necesita alta precisión de costas.



Chatbot con IA local (api_bot.php + Ollama)

IA Local Backend

El asistente de viajes del mapa llama a `api_bot.php` enviando la pregunta del usuario y recibe una recomendación de país junto con información enriquecida (bandera, capital, región, población). El backend usa un modelo de lenguaje local (tipo Ollama/LLM) para procesar la petición y devolver datos estructurados.

```
// chatbot.js const response = await fetch('api_bot.php', { method: 'POST',  
body: JSON.stringify({ mensaje: message }) }); // Respuesta: { status:  
'success', pais: 'Japón', datos: { flags, capital, population, region } }
```

¿Por qué IA local y no ChatGPT/OpenAI? Un modelo local no tiene coste por API call, no envía datos de usuarios a servidores externos y funciona sin conexión a internet. La privacidad del usuario se mantiene.



Three.js (Blue Marble texture vía unpkg)

Frontend

Globe.gl usa internamente Three.js para el renderizado WebGL. La textura de la Tierra (*earth-blue-marble.jpg*) se descarga de `unpkg.com/three-globe/example/img/` al cargar el mapa.

Globe.gl abstrae completamente la complejidad de Three.js. No es necesario escribir shaders ni código WebGL directamente.



Frontend – CSS y Diseño



CSS Custom (sin framework) – Sistema de diseño propio

Frontend

Todo el diseño visual está hecho con CSS puro organizado en ficheros temáticos:

- `styles.css` – Login/Register, globo decorativo, variables globales
- `map.css` – Layout del mapa, topbar, panel lateral, modales, carrusel
- `profile.css` – Página de perfil de usuario
- `dashboard-admin.css` – Panel de administración
- `chatbot.css` – Widget flotante del chatbot

Usa **CSS Custom Properties** (variables CSS) para el sistema de colores (`--accent: #7f5cff`, `--bg: #060816`), efectos glassmorphism con `backdrop-filter: blur()`, y diseño oscuro (*dark theme*) como estética base.

¿Por qué CSS propio y no Bootstrap o Tailwind? Bootstrap impone estilos genéricos que habrían requerido mucho override para lograr el diseño oscuro y futurista de MONAR. Tailwind habría añadido complejidad al build. Con CSS puro se tiene control total del diseño y el resultado es único.



Vanilla JavaScript (ES2022+)

Frontend

Todo el JavaScript del cliente es vanilla (sin React, Vue ni Angular). Organizado en módulos:

- `map.js` – Lógica del globo, búsqueda de países/usuarios, modales, fotos, reseñas
- `chatbot.js` – Clase Chatbot con el widget de asistente de viajes
- `profile.js` – Gestión del perfil, cambio de contraseña, subida de avatar
- `admin-dashboard.js` – Panel de administración con tabs, búsquedas y CRUD
- `app.js` – Inicialización del globo decorativo en login/register

Usa **Fetch API**, **async/await**, **ES6 classes**, **template literals** y **Promise.all** para carga paralela de datos.

¿Por qué Vanilla JS y no React/Vue? Para la complejidad de este proyecto no se justifica el overhead de un framework SPA. Las páginas son PHP server-rendered y el JS añade interactividad. React añadiría un build step, virtual DOM, y curva de aprendizaje innecesaria para lo que se necesita.



Paquetes Symfony adicionales (composer.json)

Paquete

Para qué se usa

nelmio/cors-bundle	Configurar cabeceras CORS para permitir peticiones desde el frontend al backend en desarrollo
symfony/security-bundle	Incluido por Symfony esqueleto; no se usa activamente en el sistema de auth (se usa sesiones PHP nativas)
symfony/mailer	Paquete de Symfony para envío de emails (disponible para futuras funcionalidades de recuperación de contraseña)
symfony/validator	Validación de datos con annotations en las entidades Doctrine
symfony/http-client	Cliente HTTP de Symfony para peticiones a APIs externas desde el backend
symfony/intl	Internacionalización (nombres de países, locales, monedas)
twig/twig	Motor de plantillas. Solo se usa en templates/base.html.twig (el resto son PHP puro)
monolog-bundle	Sistema de logging para errores y eventos del servidor
symfony/asset-mapper	Mapeo de assets frontend (CSS, JS) sin necesidad de Webpack/Vite
symfony/stimulus-bundle	Stimulus (JS framework ligero de Hotwire); incluido en el esqueleto Symfony
symfony/ux-turbo	Turbo Streams de Hotwire; incluido en el esqueleto Symfony

Nota importante para la defensa: Muchos paquetes Symfony (mailer, notifier, messenger, stimulus, ux-turbo, twig) fueron instalados automáticamente al crear el proyecto con symfony/skeleton. No todos se usan activamente en producción — el núcleo real del proyecto es PHP nativo + PDO + MySQL.

2. Esquema de Base de Datos

La base de datos **monuar_db** tiene las siguientes tablas, todas con motor **InnoDB**, charset **utf8mb4** y claves foráneas con **ON DELETE CASCADE**.

Tabla	Propósito	Campos clave
usuario	Usuarios del sistema	id, nombre_usuario, correo_electronico, contraseña_hash, biografia, avatar_url, fecha_registro, es_admin
pais	Catálogo de países	id, nombre, continente, codigo_iso (numérico ISO 3166-1)
visita_pais	Qué países ha marcado como visitados cada usuario	id, usuario_id (FK), pais_id (FK), fecha_visita
publicacion	Fotos subidas por un usuario para un país	id, usuario_id (FK), pais_id (FK), imagen_url, descripcion, fecha_publicacion
resena	Reseñas de países (valoraciones 1-5 estrellas)	id, usuario_id (FK), pais_id (FK), titulo, contenido, puntuacion, fecha_resena
likes	Likes de usuarios sobre fotos (publicaciones)	id, usuario_id (FK), publicacion_id (FK), fecha
usuario_seguidor	Relación de seguimiento entre usuarios (red social)	id, seguidor_id (FK→usuario), seguido_id (FK→usuario), fecha_creacion
mapa_usuario	Configuración personalizada del mapa de cada usuario	id, usuario_id (FK, UNIQUE), estilo_color, ultima_actualizacion
conversacion_chat	Sesiones de conversación con el chatbot IA	id, usuario_id (FK), fecha_inicio
mensaje_chat	Mensajes individuales de cada conversación con el bot	id, conversacion_chat_id (FK), rol, contenido, fecha_envio

messenger_messages	Cola de mensajes de Symfony Messenger (infraestructura)	id, body, headers, queue_name, created_at, available_at
---------------------------	---	--

Aspectos destacables del diseño de BD

Tabla usuario_seguidor – Creación dinámica

Esta tabla **no se crea via migración Doctrine**, sino dinámicamente en el primer uso mediante `ensureUserFollowTable()` en `config/monar_database.php`. Incluye:

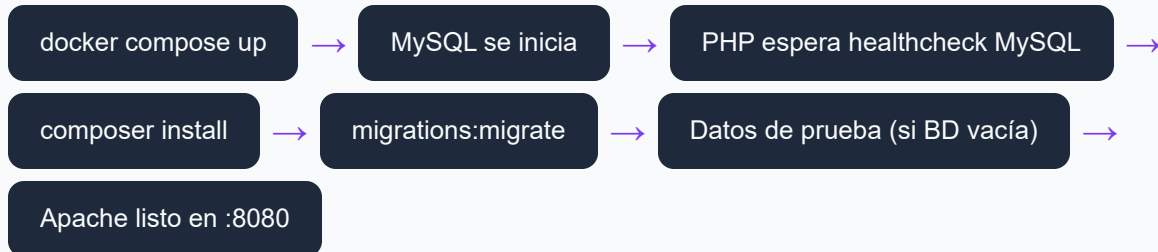
- UNIQUE KEY (seguidor_id, seguido_id) — evita follows duplicados
- CHECK (seguidor_id ≠ seguido_id) — un usuario no puede seguirse a sí mismo
- Índices en ambas columnas para búsquedas rápidas de seguidores/seguídos

Código ISO numérico

Los países se identifican con el código **ISO 3166-1 numérico** (3 dígitos: España='724', Francia='250', etc.) porque ese es el formato que usa el dataset World-Atlas en [Globe.gl](#). Esto permite el cruce directo entre los datos de la BD y los polígonos del mapa sin conversión adicional.

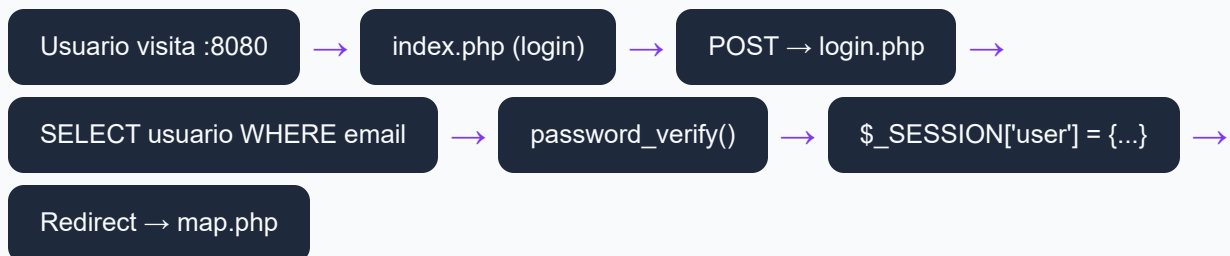
3. Funcionamiento Completo de la Web

3.1 Arranque del sistema (Docker)



El script `docker-entrypoint.sh` automatiza todo este proceso. Si la tabla `usuario` está vacía, carga el archivo `database/init_test_data.sql` con países y usuarios de ejemplo.

3.2 Registro e inicio de sesión



- **index.php:** Muestra el formulario de login con el globo 3D decorativo en el lado izquierdo
- **login.php:** Procesa el POST, consulta la BD con prepared statement, verifica con `password_verify()`, crea sesión y redirige
- **register_form.php:** Formulario de registro (misma estética con globo)
- **register.php:** Valida datos (email único, mínimos de longitud), hashlea contraseña con `password_hash()`, inserta usuario
- **logout.php:** `session_destroy()` y redirect a index

3.3 El mapa principal (map.php)

Esta es la pantalla central de la aplicación. Al cargar `map.php`:

1. PHP verifica la sesión. Si no hay sesión → redirect a `index.php`
2. PHP determina si estamos viendo nuestro propio mapa o el de otro usuario (`?view_user=ID`)
3. Se renderiza el HTML con el layout: topbar + globo 3D + panel lateral
4. El JS (`map.js`) carga en paralelo con `Promise.all`:

- `api/get_visited_countries.php` → países visitados del usuario (de MySQL)
- `unpkg.com/world-atlas@2/countries-110m.json` → polígonos de todos los países del mundo

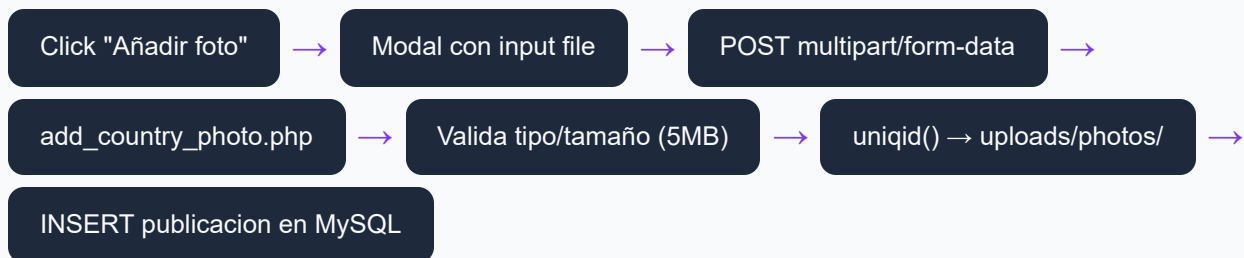
5. Globe.gl renderiza el globo y pinta cada país según si está en la lista de visitados

6. Se activa la rotación automática del globo

3.4 Interacción con el globo

Acción del usuario	Qué ocurre
Click en un país visitado	Panel lateral muestra nombre, continente, fecha de visita. Se cargan las fotos (<code>get_country_photos.php</code>) y reseñas (<code>get_country_reviews.php</code>) del usuario para ese país. Aparecen botones "Añadir foto" y "Añadir reseña".
Click en un país no visitado	Panel lateral muestra el nombre y un botón "Marcar como visitado". Click → POST a <code>add_visited_country.php</code> → INSERT en <code>visita_pais</code> → reload del mapa.
Búsqueda de país	Input filtra en tiempo real contra <code>api/get_countries.php</code> (lista cargada al inicio). Al seleccionar, el globo vuela al país con animación (1200ms).
Eliminar país visitado	Botón en el panel → POST a <code>remove_visited_country.php</code> → DELETE en <code>visita_pais</code> → reload.
Drag / Scroll	Globe.gl gestiona la rotación manual y el zoom con Three.js OrbitControls.

3.5 Fotos de países



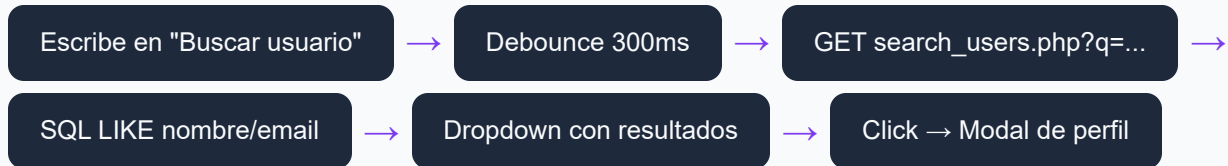
- Formatos permitidos: JPG, PNG, GIF, WEBP
- Tamaño máximo: 5MB
- Nombre único generado con `uniqid('photo_' . $user_id . '_')` para evitar colisiones
- La URL relativa se guarda en la BD; el archivo físico va a `public/uploads/photos/`
- Cada foto puede recibir likes (toggle) via `api/toggle_photo_like.php`

3.6 Reseñas de países

El modal de reseña permite escribir título, contenido y puntuar de 1 a 5 estrellas. El POST va a `add_country_review.php` que:

- Lee el body como JSON (`php://input`)
- Verifica que el usuario tenga ese país en `visita_pais` (solo se puede reseñar lo que se ha visitado)
- INSERT en tabla `resena`

3.7 Red social – Búsqueda de usuarios y seguimiento



El modal de perfil de usuario muestra:

- Nombre, email, fecha de registro
- Estadísticas: países visitados, reseñas, continentes
- Contador de seguidores/seguídos
- Botón "Seguir/Dejar de seguir" → POST a `toggle_follow_user.php`
- Botón "Visitar su mapa" → redirige a `map.php?view_user=ID`

Modo vista de otro usuario: Cuando se accede a `map.php?view_user=ID`, el globo muestra los países visitados de ese usuario (en morado) en lugar de los del usuario actual. Se ocultan los botones de añadir/eliminar pero se pueden ver sus fotos y reseñas.

3.8 Perfil del usuario (`profile.php`)

Página de gestión de la cuenta personal. Permite:

Acción	Endpoint	Descripción
Cambiar contraseña	<code>api/change_password.php</code>	Verifica contraseña actual, valida nueva, hashea y actualiza
Actualizar perfil	<code>api/update_profile.php</code>	Cambia nombre de usuario y/o biografía
Subir avatar	<code>api/upload_avatar.php</code>	Sube imagen de perfil (misma validación que fotos)

Eliminar cuenta	api/delete_account.php	Borra el usuario y todos sus datos (CASCADE en FK)
Ver estadísticas	api/get_user_stats.php	Países visitados, reseñas, continentes, seguidores, seguidos

Incluye un indicador visual de fortaleza de contraseña calculado en tiempo real en JavaScript.

3.9 Panel de administración (dashboard.php)

Solo accesible a usuarios con es_admin = 1 en la BD. Consta de 3 tabs:

Tab	Endpoint(s)	Funcionalidad
Usuarios	admin_get_users.php / admin_delete_user.php	Listado de todos los usuarios, búsqueda en tiempo real, eliminación
Reseñas	admin_get_reviews.php / admin_delete_review.php	Moderación de reseñas, eliminación de contenido inapropiado
Fotos	admin_get_photos.php / admin_delete_photo.php	Moderación de fotos subidas por usuarios

El header muestra estadísticas globales del sistema (total usuarios, visitas, reseñas, fotos) obtenidas de admin_statistics.php.

Cada endpoint admin verifica que es_admin = 1 en la BD, no solo en sesión, para evitar escalada de privilegios.

3.10 Chatbot asistente de viajes

Widget flotante disponible en el mapa. El usuario escribe un destino o pregunta de viaje y el bot responde con:

- Nombre del país recomendado
- Tarjeta visual con bandera, capital, región y población



La clase JavaScript Chatbot crea dinámicamente el HTML del widget, gestiona el estado abierto/cerrado, muestra un indicador "escribiendo..." mientras espera la respuesta, y renderiza las tarjetas de países con flags y datos.

4. Arquitectura del Proyecto

Patrón arquitectónico: PHP Native API + SPA-like Frontend

MONAR usa un patrón **híbrido**:

- **Server-side rendering** con PHP para las páginas principales (map.php, profile.php, dashboard.php) — PHP genera el HTML inicial con datos del usuario ya incrustados
- **API REST JSON** para todas las operaciones dinámicas: el JS del cliente llama a `api/*.php` con `fetch()` y recibe/envía JSON
- **Sesiones PHP** para autenticación — todas las páginas y endpoints verifican `$_SESSION['user']`

```
Proyecto/ ├── docker-compose.yml ← Orquestación Docker (app + db) └── backend/ ├──
Dockerfile ← php:8.4-apache + Composer ├── docker-entrypoint.sh ← Script arranque:
migraciones + datos ├── composer.json ← Dependencias PHP (Symfony, Doctrine) ├──
config/ ├── monar_database.php ← Conexión PDO + ensureUserFollowTable() ├──
migrations/ ← Migraciones Doctrine (versionado BD) ├── Version20260413122830.php
└── public/ ← DocumentRoot Apache (única carpeta expuesta) ├── index.php ← Login ├──
register_form.php ← Formulario registro ├── register.php ← Procesa registro ├──
login.php ← Procesa login ├── logout.php ← Destruye sesión ├── map.php ← Mapa
principal ★ ├── profile.php ← Perfil usuario ├── dashboard.php ← Admin panel ├──
api/ ← API REST (25 endpoints) ├── get_countries.php ├──
get_visited_countries.php ├── add_visited_country.php ├──
remove_visited_country.php ├── get_country_photos.php ├── add_country_photo.php
├── get_country_reviews.php ├── add_country_review.php ├──
toggle_photo_like.php ├── toggle_follow_user.php ├── search_users.php ├──
get_user_profile.php ├── get_user_stats.php ├── change_password.php ├──
update_profile.php ├── upload_avatar.php ├── delete_account.php ├──
delete_own_photo.php ├── delete_own_review.php ├── admin_*.php ← (5 endpoints
admin) ├── uploads/photos/ ← Imágenes subidas por usuarios ├── assets/ ├── css/ ← 5
hojas de estilo └── js/ ← 5 módulos JavaScript
```

¿Por qué PHP nativo para la API en lugar de Symfony MVC?

Cada endpoint de la API es un fichero PHP autocontenido: verifica sesión → conecta a BD → ejecuta query → devuelve JSON. Este patrón es directo, fácil de depurar y no añade capas de abstracción

innecesarias para el tamaño del proyecto. Symfony se reserva para lo que mejor hace: gestión de migraciones de BD.

5. Seguridad Implementada

Amenaza (OWASP)	Medida implementada	Dónde
A03 – Inyección SQL	100% prepared statements PDO con <code>PDO::ATTR_EMULATE_PREPARES = false</code> . Ninguna query construye SQL concatenando variables.	Todos los archivos PHP con BD
A02 – Fallos criptográficos	<code>password_hash(PASSWORD_DEFAULT) = bcrypt</code> con salt aleatorio. La contraseña en texto plano nunca se almacena.	<code>register.php</code> , <code>change_password.php</code>
A01 – Control de acceso roto	Verificación de sesión en cada página y endpoint API. Admin: verificación en BD (<code>es_admin</code>) además de sesión.	Todos los <code>*.php</code>
A03 – XSS	<code>htmlspecialchars()</code> en todas las salidas PHP al HTML. El JSON de las APIs no genera HTML directo.	<code>map.php</code> , <code>profile.php</code> , etc.
A04 – Diseño inseguro	Solo la carpeta <code>public/</code> está expuesta como <code>DocumentRoot</code> . Los ficheros de config (<code>monar_database.php</code> con credenciales) están fuera del <code>DocumentRoot</code> .	Dockerfile, Apache config
A05 – Subida de archivos	Validación de MIME type + extensión + tamaño (5MB). Nombre de archivo regenerado con <code>uniqid()</code> . Solo tipos imagen permitidos.	<code>add_country_photo.php</code> , <code>upload_avatar.php</code>
A07 – Fallos de autenticación	No se revela si el email existe o no en mensajes de error de login genéricos.	<code>login.php</code>
Datos propios	Al eliminar fotos/reseñas propias, se verifica que <code>usuario_id =</code>	<code>delete_own_photo.php</code> , <code>delete_own_review.php</code>

`$_SESSION['user']['id']` antes del
DELETE.

Auto-follow

La BD tiene un CHECK constraint
`seguidor_id ≠ seguido_id` y el
backend también lo verifica.

`toggle_follow_user.php`

CORS

nelmio/cors-bundle configurado para
controlar qué orígenes pueden hacer
peticiones cross-origin.

`config/packages/nelmio_cors.yaml`

6. Preguntas de Defensa del TFG

Las siguientes son las preguntas más probables que puede hacerte el tribunal. Cada respuesta está argumentada con los detalles técnicos reales del proyecto.

Sobre las tecnologías elegidas

? ¿Por qué usaste PHP y no tecnologías más modernas como Node.js o Python/Django?

PHP sigue siendo el lenguaje servidor más utilizado en la web (WordPress, Wikipedia, Facebook usan PHP). Para este proyecto es la elección ideal porque: (1) se integra nativamente con Apache sin configuración extra, (2) la imagen Docker `php:8.4-apache` tiene todo lo necesario, (3) PHP 8.4 tiene JIT compiler que lo hace competitivo en rendimiento, (4) el ecosistema con Composer y Symfony/Doctrine es muy maduro. Node.js habría requerido gestionar asincronía en el servidor de forma más compleja; Python/Django habría añadido una curva de aprendizaje mayor sin ventajas reales para este tipo de aplicación web.

? ¿Por qué Symfony si apenas lo usas? ¿No es un overhead innecesario?

Es una observación correcta: Symfony no gestiona el routing ni las vistas. Sin embargo, se usa estratégicamente para lo que mejor hace: el sistema de migraciones de Doctrine. Esto permite que cuando se hace `docker compose up`, el esquema de la base de datos se aplique automáticamente, de forma versionada y sin perder datos. Alternativas como ejecutar un script SQL manual son propensas a errores si ya existe parte del esquema. El "overhead" de Composer instalar Symfony se compensa completamente con la robustez del sistema de migraciones.

? ¿Por qué usas Globe.gl y no Google Maps o Leaflet?

Leaflet y Google Maps son mapas 2D planos. El elemento diferenciador de MONAR es precisamente el globo 3D interactivo que permite ver el mundo desde perspectiva espacial, rotarlo, hacer zoom, y ver los países visitados iluminados en el globo. Globe.gl es open source, gratuito (Google Maps tiene coste por encima de cierto número de peticiones), y permite personalizar completamente los colores, texturas y efectos de atmósfera. Ninguna librería de mapas 2D podría ofrecer la misma experiencia visual.

? ¿Por qué no usaste React, Vue o Angular para el frontend?

Las páginas de MONAR son renderizadas en servidor por PHP (server-side rendering). El JavaScript añade interactividad sobre ese HTML ya existente. Un framework SPA como React habría requerido: (1) un proceso de build (Webpack/Vite), (2) separar completamente frontend y backend en proyectos distintos, (3) gestionar el estado global de la aplicación con Redux o similar, (4) manejar el routing del lado cliente. Todo eso sin ningún beneficio funcional para este proyecto. Vanilla JS con `async/await` y la Fetch API es suficiente para la complejidad requerida y resulta en código más directo y sin dependencias.

? ¿Por qué Docker? ¿No es excesivo para un TFG?

Docker garantiza que el proyecto funciona exactamente igual en cualquier máquina. Sin Docker, para ejecutar MONAR habría que instalar PHP 8.4, configurar Apache, instalar MySQL 8.0, configurar las variables de entorno y ejecutar las migraciones manualmente. Con Docker, es un solo comando: `docker compose up`. Esto es fundamental para la evaluación del TFG (el tribunal puede ejecutarlo sin problemas) y refleja una práctica profesional real —ningún proyecto profesional se despliega sin contenedores hoy en día.

? ¿Por qué MySQL y no una base de datos NoSQL como MongoDB?

Los datos de MONAR son fuertemente relacionales: un usuario tiene visitas, cada visita pertenece a un país, cada país tiene fotos y reseñas, cada foto tiene likes, los usuarios se siguen entre sí. Este tipo de relaciones N:M y 1:N se modelan perfectamente con SQL y claves foráneas con integridad referencial. MongoDB habría implicado gestionar esas relaciones manualmente en código, sin las garantías de consistencia que da el motor InnoDB (transacciones ACID). MySQL es la elección correcta para datos relacionales.

? ¿Qué es el dataset World-Atlas y por qué lo usas?

World-Atlas es un dataset público basado en Natural Earth data que contiene los polígonos geográficos de todos los países del mundo en formato TopoJSON. Es el mismo dataset que usa D3.js para visualizaciones geográficas. Lo uso porque Globe.gl necesita los polígonos de los países para pintarlos sobre el globo. El archivo `countries-110m.json` (escala 1:110 millones) tiene resolución suficiente para un globo 3D y pesa ~100KB, lo que es muy ligero para cargar vía CDN. Cada país tiene un ID numérico ISO 3166-1 que cruzo con los códigos de mi base de datos para saber qué países colorear de morado (visitados).



Sobre la arquitectura

? ¿Cómo funciona la autenticación? ¿Es segura?

La autenticación usa sesiones PHP nativas. Al hacer login, `login.php` busca el usuario por email con un prepared statement (evita SQL injection), verifica la contraseña con `password_verify()` contra el hash bcrypt almacenado (evita guardar contraseñas en texto plano), y si es correcta crea `$_SESSION['user']` con el id, email y nombre. Cada página y cada endpoint API verifica `isset($_SESSION['user'])` al inicio. Para el admin, además de la sesión se consulta `es_admin` directamente en la BD para evitar que alguien manipule la sesión y se auto-asigne privilegios. Es un sistema estándar y seguro para una aplicación web PHP.

? ¿Cómo evitas la inyección SQL?

Mediante prepared statements PDO en el 100% de las consultas. Con PDO, el SQL y los datos viajan separados al motor MySQL: primero se envía la estructura SQL con parámetros (`:email`, `:id...`) y luego los valores. MySQL los trata siempre como datos, nunca como código SQL. Además se configura `PDO::ATTR_EMULATE_PREPARES = false` para que sean prepared statements reales del servidor MySQL, no emulados por PHP. Ejemplo: `$stmt = $pdo->prepare('SELECT * FROM usuario WHERE correo_electronico = :email')` — aunque el email contenga `' OR '1'='1'`, MySQL lo trata como texto literal.

? ¿Por qué el mapa puede ver países de otros usuarios? ¿No es un problema de privacidad?

Es una funcionalidad social deliberada, similar a Instagram o Strava donde puedes ver el perfil público de otros. La característica de red social de MONAR permite descubrir a qué países han viajado tus amigos e inspirarte para tus próximos viajes. En el "modo vista" (`map.php?view_user=ID`) solo se muestra información pública: países visitados, fotos y reseñas — no datos privados como email o contraseña. El usuario que visita otro perfil primero busca por nombre/email y accede desde el modal de perfil. En el futuro podría añadirse un sistema de privacidad (perfil público/privado).

? ¿Cómo funciona el sistema de seguimiento (followers)?

La tabla `usuario_seguidor` almacena pares (`seguidor_id`, `seguido_id`). La relación es unidireccional (como Twitter, no como Facebook): puedes seguir a alguien sin que te siga. El endpoint `toggle_follow_user.php` comprueba si ya existe la fila — si existe la elimina (unfollow), si no existe la inserta (follow). La tabla tiene un UNIQUE KEY en (`seguidor_id`, `seguido_id`) para evitar duplicados incluso en condiciones de concurrencia, y un CHECK constraint `seguidor_id ≠ seguido_id` para

prevenir que un usuario se siga a sí mismo. Esta tabla se crea dinámicamente con `CREATE TABLE IF NOT EXISTS` para no depender de una migración específica.

? ¿Cómo gestionas la subida de archivos de forma segura?

Hay cuatro capas de validación en `add_country_photo.php` y `upload_avatar.php`: (1) verificar que no hubo error en la subida (`UPLOAD_ERR_OK`), (2) validar el MIME type contra una whitelist explícita (`image/jpeg`, `image/png`, `image/gif`, `image/webp`), (3) validar que el tamaño no supere 5MB, (4) generar un nombre de archivo nuevo con `uniqid()` que incluye el `user_id` para evitar sobrescribir archivos de otros usuarios y para que nombres maliciosos no afecten al sistema de ficheros. Los archivos se guardan en `public/uploads/photos/` fuera de cualquier ruta PHP ejecutable desde web.

📖 Sobre la base de datos

? ¿Por qué usas códigos ISO numéricos y no los de 2 letras (ES, FR...)?

El dataset World-Atlas identifica cada país con su código ISO 3166-1 numérico como ID del feature GeoJSON/TopoJSON (724 para España, 250 para Francia...). Si usara códigos alpha-2, necesitaría una tabla de traducción adicional o un mapeo en código. Al almacenar directamente el código numérico en la BD, el cruce entre los polígonos del mapa y los países de la BD es directo:
`visitedByIso[feature.id]`. Solo se necesitan mapeos especiales para casos como Antártida (010→AQ) y Groenlandia (304→GL).

? ¿Qué son las migraciones y para qué sirven?

Una migración es un script PHP (generado por Doctrine) que aplica un cambio al esquema de la base de datos de forma versionada. El fichero `Version20260413122830.php` crea todas las tablas de MONAR (usuario, pais, visita_pais, publicacion, resena, likes, etc.) con sus claves foráneas. Doctrine guarda en una tabla interna (`doctrine_migration_versions`) qué migraciones se han aplicado. Cuando arranca Docker, el comando `doctrine:migrations:migrate` aplica solo las migraciones pendientes. Esto garantiza que el esquema es siempre el correcto sin importar si la BD es nueva o ya tiene datos de una versión anterior.

💡 Sobre las funcionalidades

? ¿Qué hace exactamente el chatbot? ¿Usa inteligencia artificial real?

Sí, el chatbot usa un modelo de lenguaje (LLM) ejecutado localmente. El widget JavaScript envía el mensaje del usuario a `api_bot.php` via POST JSON. El backend procesa la petición con un LLM local (tipo Ollama con un modelo como Llama o Mistral) que entiende la petición en lenguaje natural ("quiero un destino exótico en Asia"), identifica un país y devuelve la respuesta estructurada con los datos del destino (nombre, capital, región, población, bandera). Al ser un modelo local, no hay coste por petición ni se envían datos de los usuarios a terceros.

? ¿Qué mejoras futuras le harías al proyecto?

(1) **Sistema de privacidad**: perfil público/privado, la posibilidad de que la lista de viajes solo sea visible para seguidores. (2) **Feed social**: un timeline con las actividades recientes de los usuarios que sigues (nuevo país visitado, nueva foto). (3) **Mapas personalizables**: ya existe la tabla `mapa_usuario` con `estilo_color`, que permitiría cambiar el tema de colores del globo. (4) **Gamificación**: logros y badges por visitar X países, todos los continentes, etc. (5) **Exportar el mapa** como imagen o PDF para compartir. (6) **Búsqueda por continente** con filtros avanzados.

? ¿Cómo escalaría el proyecto si tuviera miles de usuarios?

Con la arquitectura actual (PHP + MySQL en Docker), el escalado vertical (más CPU/RAM) es inmediato. Para escalado horizontal: (1) añadir un balanceador de carga delante de múltiples contenedores PHP (las sesiones necesitarían moverse a Redis para ser compartidas), (2) MySQL con réplicas de lectura para las queries más frecuentes (`get_visited_countries`, `get_countries`), (3) un CDN para servir las imágenes de `uploads/photos/` en lugar del servidor PHP, (4) caché de la lista de países en Redis (raramente cambia). La separación de API y frontend ya preparada facilita este escalado.

? ¿Cómo se diferencia MONAR de otras apps similares como Polarsteps o Been?

(1) **Globo 3D interactivo**: Polarsteps usa un mapa 2D. MONAR tiene un globo renderizado en WebGL que el usuario puede rotar y manipular en 3D. (2) **Red social integrada**: puedes buscar otros viajeros, ver su mapa en vivo y seguirlos. (3) **Fotos y reseñas por país**: cada país visitado puede tener un álbum de fotos y reseñas con valoración. (4) **Chatbot IA local**: asistente de viajes para recomendaciones. (5) **Open source y auto-hospedado**: cualquiera puede desplegar MONAR en su servidor.

? ¿Por qué usas PDO estático (singleton) para la conexión a la BD?

La función `getDBConnection()` usa una variable static `$pdo = null` que actúa como singleton de la conexión. PHP crea una nueva instancia del proceso por cada petición HTTP, por lo que la conexión se establece una vez por petición (no hay riesgo de conexiones acumuladas). El patrón singleton evita abrir

múltiples conexiones dentro de la misma petición si varios archivos llaman a `getDBConnection()`. MySQL tiene un límite de conexiones simultáneas; este patrón garantiza que una petición usa exactamente una conexión.

? ¿Cuál es el papel de las entidades Doctrine si no usas el ORM para las queries?

Las entidades Doctrine en `src/Entity/` sirven como **modelo de datos** para generar las migraciones automáticamente con `doctrine:migrations:diff`. Al modificar una entidad (añadir un campo, cambiar una relación), Doctrine puede generar automáticamente el SQL necesario para actualizar la BD. Las queries en sí se hacen con PDO nativo para tener control total del SQL, evitar el overhead del ORM y poder usar queries complejas con JOINS directamente. Es un uso pragmático: Doctrine gestiona el esquema, PDO gestiona los datos.

Resumen Visual del Stack

Capa	Tecnología	Versión	Propósito
Backend	PHP	8.4	Lenguaje servidor, API REST, sesiones
	PDO	nativo	Acceso seguro a MySQL (prepared statements)
	Symfony Framework	8.0	Migraciones DB vía Doctrine Migrations
	Doctrine ORM	^3.6	Entidades y generación de migraciones
	Apache	2.4	Servidor HTTP con mod_rewrite
Base de datos	MySQL	8.0	Base de datos relacional (InnoDB + utf8mb4)
Frontend	Vanilla JavaScript	ES2022+	Interactividad, Fetch API, clases, async/await
	Globe.gl	latest	Globo 3D WebGL interactivo
	TopoJSON client	incluido	Parseo de polígonos geográficos
	CSS custom	-	Sistema de diseño oscuro propio (5 hojas)
	Three.js	incluido en Globe.gl	Renderizado WebGL (abstracción)
APIs externas	World-Atlas @2	v2	Polígonos TopoJSON de países del mundo
	LLM local (Ollama)	-	Chatbot asistente de viajes con IA

Infraestructura	Docker	-	Contenedorización (app + db)
	Docker Compose	v3	Orquestación multi-contenedor
Herramientas	Composer	v2	Gestor de dependencias PHP
Seguridad	bcrypt + PDO prepared + htmlspecialchars	-	Contra OWASP Top 10 (SQLi, XSS, auth)

Consejo para la defensa: Si te preguntan por qué elegiste X en lugar de Y, siempre argumenta en términos de: (1) adecuación al problema, (2) simplicidad vs. complejidad, (3) coste/beneficio de aprendizaje, (4) estándares del sector. Nunca digas "lo elegí porque lo conocía" sin añadir una razón técnica.